

Ciclo-de-Vida-de-un-Hilo.pdf



JesusLagares



Programación Concurrente y de Tiempo Real



3º Grado en Ingeniería Informática



**Escuela Superior de Ingeniería
Universidad de Cádiz**



[Accede al documento original](#)

antes



**Descarga sin publi
con 1 coin**



Después

WUOLAH



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Ciclo de vida de un hilo



El *ciclo de vida* de un hilo describe los distintos **estados** por los que pasa desde que se crea hasta que termina su ejecución.

Explicación

El *ciclo de vida* de un hilo describe los distintos **estados** por los que pasa desde que se crea hasta que termina su ejecución.

Estado NEW (Nuevo)

Cuando hacemos:

```
Thread t = new Thread(miRunnable);
```

el hilo está en estado **New**.

Características:

- Es una **entidad pasiva**, igual que cualquier otro objeto creado con `new`.
- **No está registrado** en el planificador → NO puede ejecutarse.
- **No consume ciclos de CPU**.
- Aún no existe como "hilo real", solo como objeto Thread.

⚠ Si llamas a `run()` en este estado, **NO** se crea concurrencia: el código se ejecuta en el hilo actual.

Estado RUNNABLE (Listo/Ejecutable)

Entramos aquí **al invocar** `start()` :

```
t.start();
```

Esto es lo que convierte el objeto en un *hilo real*.

En este estado:

- En Java, el uso de `start` es imprescindible si queremos que el hilo pase a este estado (salvo cuando usamos ejecutores). En C++ es automática.
- El hilo **ya es elegible** para ejecutarse.
- **Todavía no está ejecutándose**: está esperando que el planificador le dé CPU. Por lo que en este estado NO CONSUME CICLOS DE CPU.
- La JVM decidirá *cuándo* invocará realmente a `run()`.

⚠ Importante:

En los *executors*, este paso es automático. El programador NO invoca `start()`; lo hace internamente el pool. El ejecutor internamente **se encarga de crear o reutilizar los hilos** y gestiona el lanzamiento (incluyendo la llamada a `start()` si fuera necesario, o activando un hilo ya existente en el pool) para llevar la tarea al estado *Listo*

Estado RUNNING (En ejecución)

El hilo llega aquí cuando el planificador del SO le otorga **tiempo de CPU**.

En este estado:

- Su método `run()` se está ejecutando realmente.
- El hilo **consume ciclos de reloj**.
- En cualquier momento puede:
 - volver a RUNNABLE (por expulsión del planificador),
 - bloquearse,
 - terminar.
- Este es un estado cuyo paso es automático, ya que será el planificador el que seleccione al hilo para ejecutar sus instrucciones.

La transición RUNNABLE → RUNNING → RUNNABLE ocurre constantemente y la gestiona el **planificador del SO**, no el programador.

Dentro de este estado, a los profesores les gusta incluir la ESPERA OCUPADA:

Espera ocupada (*busy waiting*)

- El hilo **sigue consumiendo CPU**, aunque no progresa.
- Ocurre cuando un hilo **gira en un bucle** esperando una condición:

```
while(condicionNoCumplida) {  
    // Busy wait → consume CPU inútilmente  
}
```

No aparece como estado explícito en Java, pero **conceptualmente** es un bloqueo que desgasta CPU.

MUY IMPORTANTE ESTO

A pesar de que conceptualmente se le considera un estado de bloqueo, la realidad es que en la asignatura lo que entendemos por bloqueo es solo el estado B), la **ESPERA BLOQUEADA**.

Para los profesores, si el hilo está consumiendo ciclos de reloj, su estado será RUNNING.

Estado BLOQUEADO / ESPERA

Es un estado donde **el hilo no puede continuar**, ya sea porque espera un evento o un recurso.

👉 Aquí hay dos formas de bloquearse:

A) Espera ocupada (*busy waiting*)

Se ha definido arriba.

B) Espera bloqueada (Blocked / Waiting)

Aquí el hilo **NO consume CPU**.

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali oohh
esto con 1 coin me
lo quito yo...

WUOLAH

Puede llegar a este estado por:

- **Esperar un cerrojo** (`synchronized`):
 - si otro hilo posee el lock, este pasa a *Blocked*.
- **Llamar a** `wait()` sobre un monitor (cola wait-set).
- **Llamar a** `join()`, esperando a que otro hilo termine.
- **Dormir** (`sleep()` → *Timed Waiting*).
- **Esperar una condición** con monitores o Locks.

El planificador solo reactivará el hilo cuando la causa del bloqueo termine:

- alguien libera un cerrojo,
- alguien hace `notify()` / `notifyAll()`,
- termina el tiempo de `sleep()`,
- la tarea del `join()` concluye.

Estado TERMINATED (Terminado)

El hilo entra aquí cuando:

- finaliza su método `run()` de forma natural, o
- se produce una excepción no capturada.

En este estado:

- **no consume CPU**,
- **no puede reiniciarse** (llamar a `start()` de nuevo provoca error),
- el hilo ha completado su ciclo.

Esquema Visual



